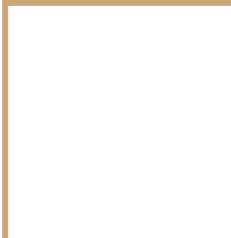


MODULE II





Arduino Basics: Development Boards-Arduino Uno, Node MCU, Arduino IDE, General Program Structure, Basic data types, variables & constants, Operators, control statements, loops, functions, time functions, arrays.



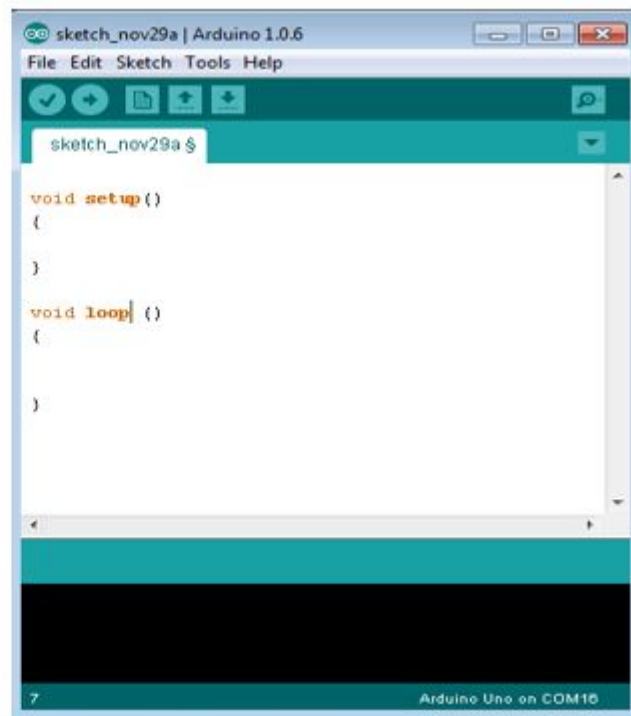
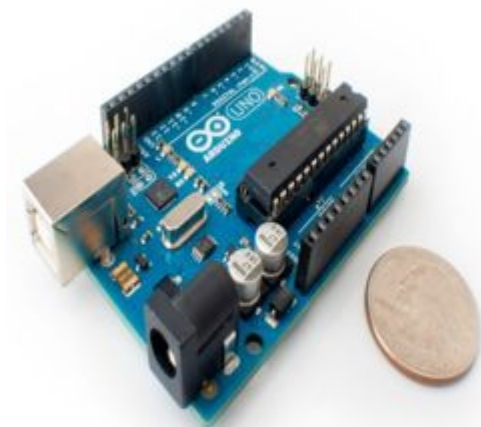
Arduino

Arduino is a prototype platform (open-source) based on an easy-to-use hardware and software.

It consists of a circuit board, which can be programmed (referred to as a microcontroller) and a ready-made software called Arduino IDE (Integrated Development Environment), which is used to write and upload the computer code to the physical board.

The key features are –

- Arduino boards are able to read analog or digital input signals from different sensors and turn it into an output such as activating a motor, turning LED on/off, connect to the cloud and many other actions.
- You can control your board functions by sending a set of instructions to the microcontroller on the board via Arduino IDE (referred to as uploading software).
- Unlike most previous programmable circuit boards, Arduino does not need an extra piece of hardware (called a programmer) in order to load a new code onto the board. You can simply use a USB cable.
- Additionally, the Arduino IDE uses a simplified version of C++, making it easier to learn to program.
- Finally, Arduino provides a standard form factor that breaks the functions of the micro-controller into a more accessible package.



Board Types

Various kinds of Arduino boards are available depending on different microcontrollers used. However, all Arduino boards have one thing in common: they are programmed through the Arduino IDE.

The differences are based on the number of inputs and outputs (the number of sensors, LEDs, and buttons you can use on a single board), speed, operating voltage, form factor etc. Some boards are designed to be embedded and have no programming interface (hardware), which you would need to buy separately. Some can run directly from a 3.7V battery, others need at least 5V.

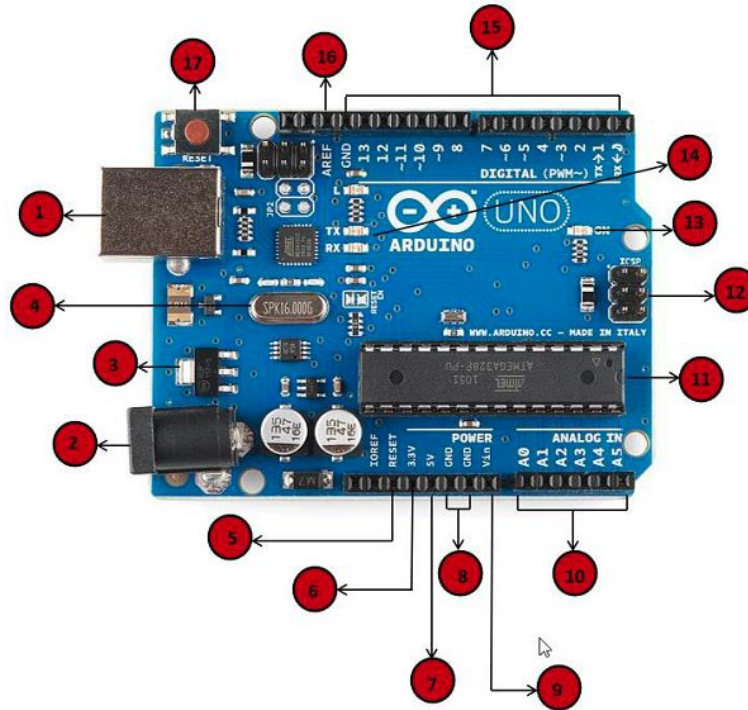
Arduino boards based on ATMEGA328 microcontroller

Board Name	Operating Volt	Clock Speed	Digital I/o	Analog Inputs	PWM	UART	Programming Interface
Arduino Uno R3	5V	16MHz	14	6	6	1	USB via ATmega16U2
Arduino Uno R3 SMD	5V	16MHz	14	6	6	1	USB via ATmega16U2
Red Board	5V	16MHz	14	6	6	1	USB via FTDI
Arduino Pro 3.3v/8 MHz	3.3V	8MHz	14	6	6	1	FTDI-Compatible Header
Arduino Pro 5V/16MHz	5V	16MHz	14	6	6	1	FTDI-Compatible Header
Arduino mini 05	5V	16MHz	14	8	6	1	FTDI-Compatible Header
Arduino Pro mini 3.3v/8mhz	3.3V	8MHz	14	8	6	1	FTDI-Compatible Header
Arduino Pro mini 5v/16mhz	5V	16MHz	14	8	6	1	FTDI-Compatible Header
Arduino Ethernet	5V	16MHz	14	6	6	1	FTDI-Compatible Header
Arduino Fio	3.3V	8MHz	14	8	6	1	FTDI-Compatible Header
LilyPad Arduino 328 main board	3.3V	8MHz	14	6	6	1	FTDI-Compatible Header
LilyPad Arduino simple board	3.3V	8MHz	9	4	5	0	FTDI-Compatible Header

Arduino boards based on ATMEGA32u4 microcontroller

Board Name	Operating Volt	Clock Speed	Digital I/o	Analog Inputs	PWM	UART	Programming Interface
Arduino Leonardo	5V	16MHz	20	12	7	1	Native USB
Pro micro 5V/16MHz	5V	16MHz	14	6	6	1	Native USB
Pro micro 3.3V/8MHz	5V	16MHz	14	6	6	1	Native USB

Arduino - Board Description



Arduino - Board Description

1. **Power USB** : Arduino board can be powered by using the USB cable from your computer. All you need to do is connect the USB cable to the USB connection (1).
2. **Power (Barrel Jack)** : Arduino boards can be powered directly from the AC mains power supply by connecting it to the Barrel Jack (2).
3. **Voltage Regulator** : The function of the voltage regulator is to control the voltage given to the Arduino board and stabilize the DC voltages used by the processor and other elements.
4. **Crystal Oscillator** : The crystal oscillator helps Arduino in dealing with time issues. How does Arduino calculate time? The answer is, by using the crystal oscillator. The number printed on top of the Arduino crystal is 16.000H9H. It tells us that the frequency is 16,000,000 Hertz or 16 MHz.
5. **Arduino Reset** : You can reset your Arduino board, i.e., start your program from the beginning. You can reset the UNO board in two ways. First, by using the reset button (17) on the board. Second, you can connect an external reset button to the Arduino pin labelled RESET (5).

(6,7,8,9) Pins (3.3, 5, GND, Vin)

- 3.3V (6) – Supply 3.3 output volt
- 5V (7) – Supply 5 output volt
- Most of the components used with Arduino board works fine with 3.3 volt and 5 volt.
- GND (8)(Ground) – There are several GND pins on the Arduino, any of which can be used to ground your circuit.
- Vin (9) – This pin also can be used to power the Arduino board from an external power source, like AC mains power supply.

Arduino - Board Description

10. Analog pins: The Arduino UNO board has six analog input pins A0 through A5. These pins can read the signal from an analog sensor like the humidity sensor or temperature sensor and convert it into a digital value that can be read by the microprocessor.

11. Main microcontroller :Each Arduino board has its own microcontroller (11). You can assume it as the brain of your board. The main IC (integrated circuit) on the Arduino is slightly different from board to board. The microcontrollers are usually of the ATMEL Company. You must know what IC your board has before loading up a new program from the Arduino IDE. This information is available on the top of the IC. For more details about the IC construction and functions, you can refer to the data sheet.

12. ICSP pin : Mostly, ICSP (12) is an AVR, a tiny programming header for the Arduino consisting of MOSI, MISO, SCK, RESET, VCC, and GND. It is often referred to as an SPI (Serial Peripheral Interface), which could be considered as an "expansion" of the output. Actually, you are slaving the output device to the master of the SPI bus.

13. Power LED indicator :This LED should light up when you plug your Arduino into a power source to indicate that your board is powered up correctly. If this light does not turn on, then there is something wrong with the connection.

14. TX and RX LEDs : On your board, you will find two labels: TX (transmit) and RX (receive). They appear in two places on the Arduino UNO board. First, at the digital pins 0 and 1, to indicate the pins responsible for serial communication. Second, the TX and RX led (13). The TX led flashes with different speed while sending the serial data. The speed of flashing depends on the baud rate used by the board. RX flashes during the receiving process.

15. Digital I/O :The Arduino UNO board has 14 digital I/O pins (15) (of which 6 provide PWM (Pulse Width Modulation) output. These pins can be configured to work as input digital pins to read logic values (0 or 1) or as digital output pins to drive different modules like LEDs, relays, etc. The pins labeled ~ can be used to generate PWM.

16. AREF : AREF stands for Analog Reference. It is sometimes, used to set an external reference voltage (between 0 and 5 Volts) as the upper limit for the analog input pins.

Arduino - Installation

Step 1 – First you must have your Arduino board (you can choose your favorite board) and a USB cable.

Step 2 – Download Arduino IDE Software.

Step 3 – Power up your board.

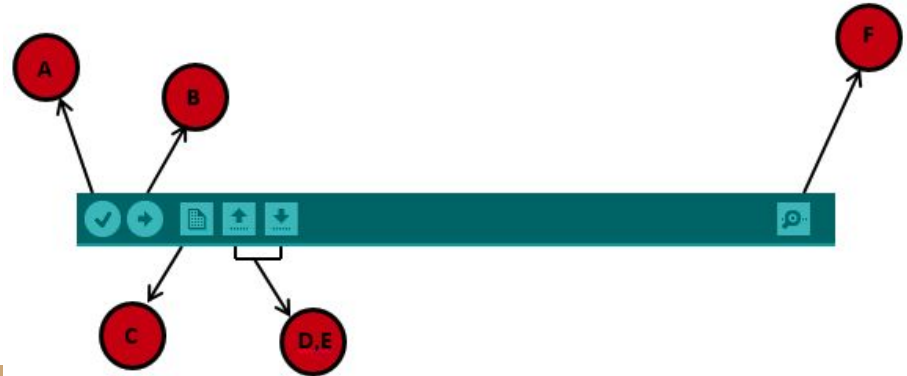
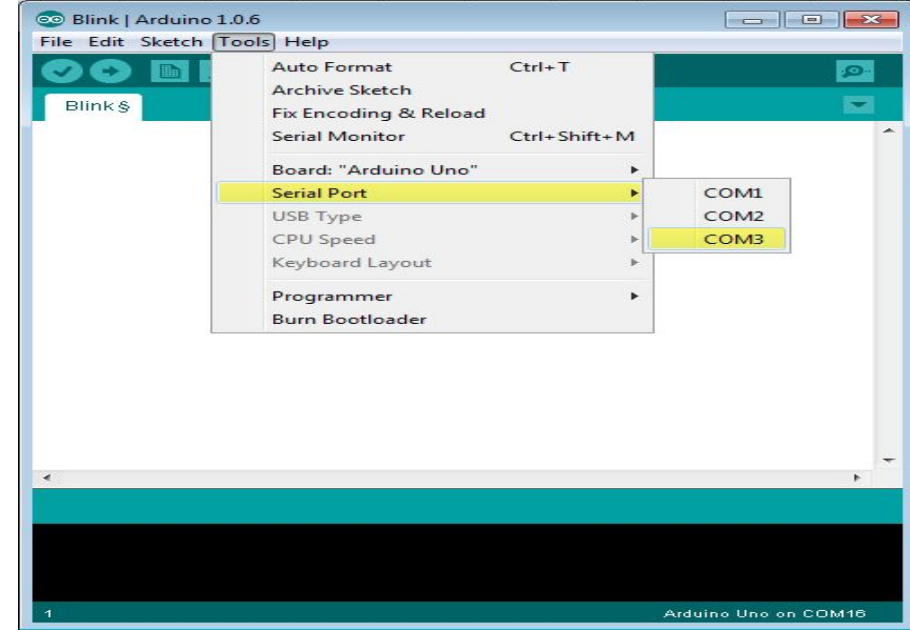
Step 4 – Launch Arduino IDE.

Step 5 – Open your first project.

Step 6 – Select your Arduino board.

Step 7 – Select your serial port.

Step 8 – Upload the program to your board.



Arduino - Program Structure

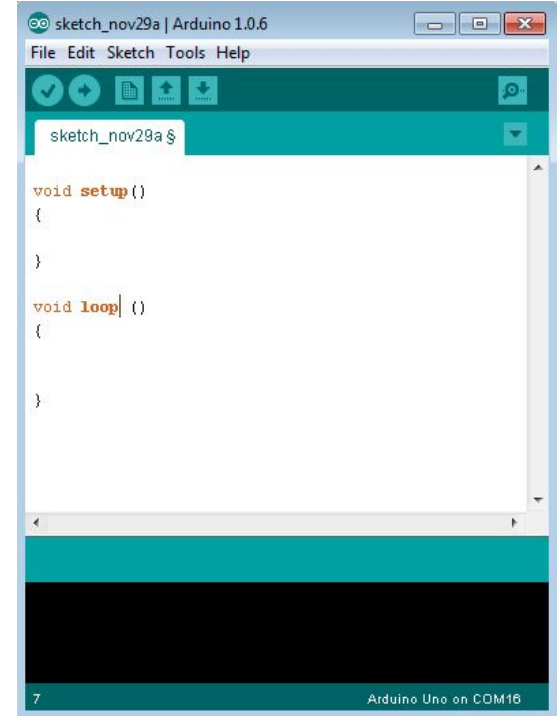
Sketch – The first new terminology is the Arduino program called **sketch**.

Structure

Arduino programs can be divided in three main parts: **Structure**, **Values** (variables and constants), and **Functions**. In this tutorial, we will learn about the Arduino software program, step by step, and how we can write the program without any syntax or compilation error.

Software structure consist of two main functions –

- Setup() function
- Loop() function



Void setup () {

}

- **PURPOSE** – The **setup()** function is called when a sketch starts. Use it to initialize the variables, pin modes, start using libraries, etc. The setup function will only run once, after each power up or reset of the Arduino board.
- **INPUT** – -
- **OUTPUT** – -
- **RETURN** – -

Void Loop () {

}

- **PURPOSE** – After creating a **setup()** function, which initializes and sets the initial values, the **loop()** function does precisely what its name suggests, and loops consecutively, allowing your program to change and respond. Use it to actively control the Arduino board.
- **INPUT** – -
- **OUTPUT** – -
- **RETURN** – -

Arduino - Data Types

The data types that you will use during Arduino programming.

void	Boolean	char	Unsigned char	byte	int	Unsigned int	word
long	Unsigned long	short	float	double	array	String-cha r array	String-obje ct

Boolean	A Boolean holds one of two values, true or false. Each Boolean variable occupies one byte of memory.	Boolean val = false ; // declaration of variable with type boolean and initialize it with false boolean state = true ; // declaration of variable with type boolean and initialize it with true
Char	A data type that takes up one byte of memory that stores a character value. Character literals are written in single quotes like this: 'A' and for multiple characters, strings use double quotes: "ABC". However, characters are stored as numbers. You can see the specific encoding in the ASCII chart . This means that it is possible to do arithmetic operations on characters, in which the ASCII value of the character is used. For example, 'A' + 1 has the value 66, since the ASCII value of the capital letter A is 65.	Char chr_a = a ; // declaration of variable with type char and initialize it with character a Char chr_c = 97 ; // declaration of variable with type char and initialize it with character 97
unsigned char	Unsigned char is an unsigned data type that occupies one byte of memory. The unsigned char data type encodes numbers from 0 to 255.	Unsigned Char chr_y = 121 ; // declaration of variable with type Unsigned char and initialize it with character y
Byte	A byte stores an 8-bit unsigned number, from 0 to 255.	byte m = 25 ; // declaration of variable with type byte and initialize it with 25
int	Integers are the primary data-type for number storage. int stores a 16-bit (2-byte) value. This yields a range of -32,768 to 32,767 (minimum value of -2^{15} and a maximum value of $(2^{15}) - 1$). The int size varies from board to board. On the Arduino Due, for example, an int stores a 32-bit (4-byte) value. This yields a range of -2,147,483,648 to 2,147,483,647 (minimum value of -2^{31} and a maximum value of $(2^{31}) - 1$).	int counter = 32 ; // declaration of variable with type int and initialize it with 32

Unsigned int	Unsigned ints (unsigned integers) are the same as int in the way that they store a 2 byte value. Instead of storing negative numbers, however, they only store positive values, yielding a useful range of 0 to 65,535 ($2^{16} - 1$). The Due stores a 4 byte (32-bit) value, ranging from 0 to 4,294,967,295 ($2^{32} - 1$).	Unsigned int counter = 60 ; // declaration of variable with type unsigned int and initialize it with 60
Word	On the Uno and other ATMEGA based boards, a word stores a 16-bit unsigned number. On the Due and Zero, it stores a 32-bit unsigned number.	word w = 1000 ;//declaration of variable with type word and initialize it with 1000
Long	Long variables are extended size variables for number storage, and store 32 bits (4 bytes), from -2,147,483,648 to 2,147,483,647.	Long velocity = 102346 ;//declaration of variable with type Long and initialize it with 102346
unsigned long	Unsigned long variables are extended size variables for number storage and store 32 bits (4 bytes). Unlike standard longs, unsigned longs will not store negative numbers, making their range from 0 to 4,294,967,295 ($2^{32} - 1$).	Unsigned Long velocity = 101006 ;// declaration of variable with type Unsigned Long and initialize it with 101006

short	A short is a 16-bit data-type. On all Arduinos (ATMega and ARM based), a short stores a 16-bit (2-byte) value. This yields a range of -32,768 to 32,767 (minimum value of -2^{15} and a maximum value of $(2^{15}) - 1$).	<pre>short val = 13 ;//declaration of variable with type short and initialize it with 13</pre>
float	Data type for floating-point number is a number that has a decimal point. Floating-point numbers are often used to approximate the analog and continuous values because they have greater resolution than integers. Floating-point numbers can be as large as 3.4028235E+38 and as low as -3.4028235E+38. They are stored as 32 bits (4 bytes) of information.	<pre>float num = 1.352; //declaration of variable with type float and initialize it with 1.352</pre>
double	On the Uno and other ATMEGA based boards, Double precision floating-point number occupies four bytes. That is, the double implementation is exactly the same as the float, with no gain in precision. On the Arduino Due, doubles have 8-byte (64 bit) precision.	<pre>double num = 45.352 ;// declaration of variable with type double and initialize it with 45.352</pre>

Arduino - Variables & Constants

What is Variable Scope?

A scope is a region of the program and there are three places where variables can be declared. They are –

- Inside a function or a block, which is called **local variables**.
- In the definition of function parameters, which is called **formal parameters**.
- Outside of all functions, which is called **global variables**.

Local Variables

Variables that are declared inside a function or block are local variables. They can be used only by the statements that are inside that function or block of code. Local variables are not known to function outside their own. Following is the example using local variables –

```
Void setup () {  
}  
Void loop () {  
    int x , y ;  
    int z ;    / /Local variable declaration  
    x = 0;  
    y = 0;    // actual initialization  
    z = 10;  
}
```

Global Variables

Global variables are defined outside of all the functions, usually at the top of the program. The global variables will hold their value throughout the life-time of your program.

A global variable can be accessed by any function. That is, a global variable is available for use throughout your entire program after its declaration.

The following example uses global and local variables –

```
Int T , S ;  
float c = 0 ; Global variable declaration  
  
Void setup () {  
}  
  
Void loop () {  
  int x , y ;  
  int z ; Local variable declaration  
  x = 0 ;  
  y = 0 ; actual initialization  
  z = 10 ;  
}
```

Arduino - Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. C language is rich in built-in operators and provides the following types of operators –

- Arithmetic Operators
- Comparison Operators
- Boolean Operators
- Bitwise Operators
- Compound Operators

[Arithmetic Operators]

- `+` → Addition
- `-` → Subtraction
- `*` → Multiplication
- `/` → Division
- `%` → Modulo

[Compound Assignment Operators]

- `+=` → Add and assign
- `-=` → Subtract and assign
- `*=` → Multiply and assign
- `/=` → Divide and assign
- `%=` → Modulo and assign
- `&=` → Bitwise AND and assign
- `|=` → Bitwise OR and assign

[Comparison Operators]

- `==` → Equal to
- `!=` → Not equal to
- `<` → Less than
- `>` → Greater than
- `<=` → Less than or equal to
- `>=` → Greater than or equal to

[Boolean (Logical) Operators]

- `&&` → Logical AND
- `||` → Logical OR
- `!` → Logical NOT

[Increment/Decrement]

- `++` → Increment
- `--` → Decrement

[Bitwise Operators]

- `&` → Bitwise AND
- `|` → Bitwise OR
- `^` → Bitwise XOR
- `~` → Bitwise NOT
- `<<` → Left Shift
- `>>` → Right Shift

Arduino - Control Statements

1. **if statement**
2. **if...else statement**
3. **if...else if...else statement**
4. **switch...case statement**
5. **Ternary operator ? :**

[if Statement]

Executes block **only if** condition is true.

```
if (condition) {  
    // code runs if condition is true  
}
```


[if...else Statement]

Chooses between two blocks based on the condition.

```
if (condition) {  
    // runs if true  
} else {  
    // runs if false  
}
```

[if...else if...else Chain]

Tests **multiple conditions** in order.

```
if (condition1) {  
    // runs if condition1 is true  
} else if (condition2) {  
    // runs if condition2 is true  
} else {  
    // runs if none are true  
}
```

[switch...case Statement]

Selects code to run **based on matching value**.

```
switch (variable) {  
  
    case value1:  
  
        // code  
  
        break;  
  
    case value2:  
  
        // code  
  
        break;  
  
    default:  
  
        // code  
  
}
```

[Ternary Operator **? :**]

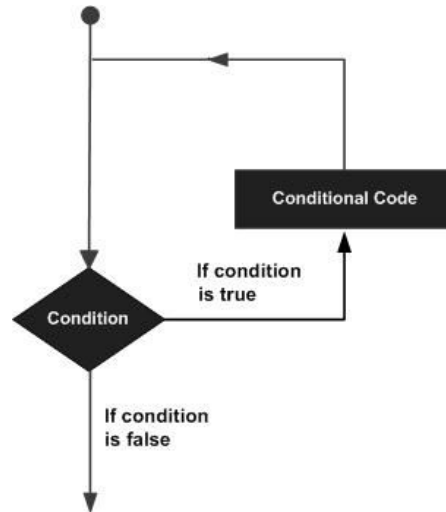
Shorthand for if-else — returns a value based on condition.

```
result = (condition) ? value_if_true : value_if_false;
```

Arduino - Loops

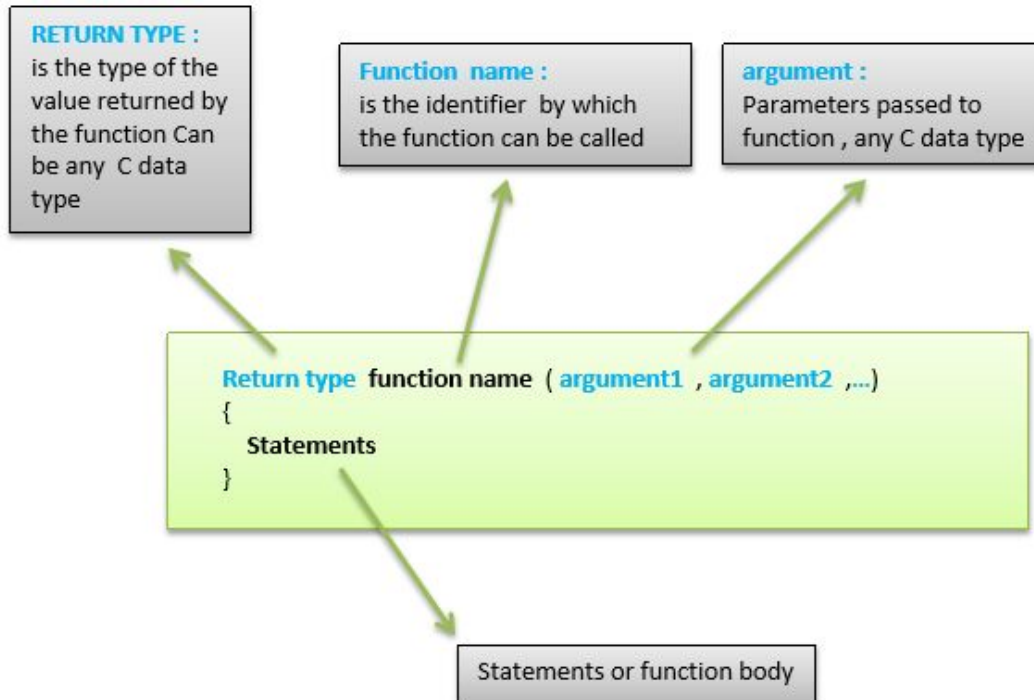
Programming languages provide various control structures that allow for more complicated execution paths.

A loop statement allows us to execute a statement or group of statements multiple times and following is the general form of a loop statement in most of the programming languages –



S.NO.	Loop & Description
1	while loop while loops will loop continuously, and infinitely, until the expression inside the parenthesis, () becomes false. Something must change the tested variable, or the while loop will never exit.
2	dowhile loop The dowhile loop is similar to the while loop. In the while loop, the loop-continuation condition is tested at the beginning of the loop before performed the body of the loop.
3	for loop A for loop executes statements a predetermined number of times. The control expression for the loop is initialized, tested and manipulated entirely within the for loop parentheses.
4	Nested Loop C language allows you to use one loop inside another loop. The following example illustrates the concept.
5	Infinite loop It is the loop having no terminating condition, so the loop becomes infinite.

Arduino - Functions



Example

```
int sum_func (int x, int y) // function declaration  
{  
    int z = 0;  
    z = x+y;  
    return z; // return the value  
}
```

```
void setup ()  
{  
    Statements // group of statements  
}
```

```
Void loop ()  
{  
    int result = 0;  
    result = Sum_func (5,6) ; // function call  
}
```



```
int sum_func (int , int ) ; // function prototype
```

```
void setup ()  
{  
  Statements // group of statements  
}
```

```
Void loop ()  
{  
  int result = 0 ;  
  result = Sum_func (5,6) ; // function call  
}
```

```
int sum_func (int x, int y) // function declaration  
{  
  int z = 0;  
  z = x+y;  
  return z; // return the value  
}
```

Arduino - Strings

Strings are used to store text.

They can be used to display text on an LCD or in the Arduino IDE Serial Monitor window.

Strings are also useful for storing the user input.

For example, the characters that a user types on a keypad connected to the Arduino.

There are two types of strings in Arduino programming –

- Arrays of characters, which are the same as the strings used in C programming.
- The Arduino String, which lets us use a string object in a sketch.

String Character Arrays

The first type of string that we will learn is the string that is a series of characters of the type **char**.

A string is an array of char variables.

A string is a special array that has one extra element at the end of the string, which always has the value of 0 (zero). This is known as a "null terminated string".

String Character Array Example

```
void setup() {  
  char my_str[6]; // an array big enough for a 5 character string  
  Serial.begin(9600);  
  my_str[0] = 'H'; // the string consists of 5 characters  
  my_str[1] = 'e';  
  my_str[2] = 'l';  
  my_str[3] = 'l';  
  my_str[4] = 'o';  
  my_str[5] = 0; // 6th array element is a null terminator  
  Serial.println(my_str);  
}  
  
void loop() {  
  
}
```

The string can be printed out to the Arduino IDE Serial Monitor window by using Serial.println() and passing the name of the string.

Alternative Way :

```
void setup() {  
    char my_str[] = "Hello";  
    Serial.begin(9600);  
    Serial.println(my_str);  
}
```

```
void loop() {  
  
}
```

Manipulating String Arrays

```
void setup() {  
  char like[] = "I like coffee and cake"; // create a string  
  Serial.begin(9600);  
  // (1) print the string  
  Serial.println(like);  
  // (2) delete part of the string  
  like[13] = 0;  
  Serial.println(like);  
  // (3) substitute a word into the string  
  like[13] = ' '; // replace the null terminator with a space  
  like[18] = 't'; // insert the new word  
  like[19] = 'e';  
  like[20] = 'a';  
  like[21] = 0; // terminate the string  
  Serial.println(like);  
}  
  
void loop() {  
  
}
```

Manipulating String Arrays

```
void setup() {  
  char like[] = "I like coffee and cake"; // create a string  
  Serial.begin(9600);  
  // (1) print the string  
  Serial.println(like);  
  // (2) delete part of the string  
  like[13] = 0;  
  Serial.println(like);  
  // (3) substitute a word into the string  
  like[13] = ' '; // replace the null terminator with a space  
  like[18] = 't'; // insert the new word  
  like[19] = 'e';  
  like[20] = 'a';  
  like[21] = 0; // terminate the string  
  Serial.println(like);  
}  
  
void loop() {  
  
}
```

Result

```
I like coffee and cake  
I like coffee  
I like coffee and tea
```

Functions to Manipulate String Arrays

```
void setup()
{
  char str[] = "This is my string"; // create a string
  char out_str[40]; // output from string functions placed here
  int num; // general purpose integer
  Serial.begin(9600);

  // (1) print the string
  Serial.println(str);

  // (2) get the length of the string (excludes null terminator)
  num = strlen(str);
  Serial.print("String length is: ");
  Serial.println(num);

  // (3) get the length of the array (includes null terminator)
  num = sizeof(str); // sizeof() is not a C string function
  Serial.print("Size of the array: ");
  Serial.println(num);

  // (4) copy a string
  strcpy(out_str, str);
  Serial.println(out_str);

  // (5) add a string to the end of a string (append)
  strcat(out_str, " sketch.");
  Serial.println(out_str);
  num = strlen(out_str);
  Serial.print("String length is: ");
  Serial.println(num);
  num = sizeof(out_str);
  Serial.print("Size of the array out_str[]: ");
  Serial.println(num);
}

void loop()
{
```


Functions to Manipulate String Arrays

```
void setup()
{
  char str[] = "This is my string"; // create a string
  char out_str[40]; // output from string functions placed here
  int num; // general purpose integer
  Serial.begin(9600);

  // (1) print the string
  Serial.println(str);

  // (2) get the length of the string (excludes null terminator)
  num = strlen(str);
  Serial.print("String length is: ");
  Serial.println(num);

  // (3) get the length of the array (includes null terminator)
  num = sizeof(str); // sizeof() is not a C string function
  Serial.print("Size of the array: ");
  Serial.println(num);

  // (4) copy a string
  strcpy(out_str, str);
  Serial.println(out_str);

  // (5) add a string to the end of a string (append)
  strcat(out_str, " sketch.");
  Serial.println(out_str);
  num = strlen(out_str);
  Serial.print("String length is: ");
  Serial.println(num);
  num = sizeof(out_str);
  Serial.print("Size of the array out_str[]: ");
  Serial.println(num);
}

void loop()
{
```

Result

This is my string

String length is: 17

Size of the array: 18

This is my string

This is my string sketch.

String length is: 25

Size of the array out_str[]: 40

Arduino - String Object

What is an Object?

An object is a construct that contains both data and functions.

A String object can be created just like a variable and assigned a value or string.

The String object contains functions (which are called "methods" in object oriented programming (OOP)) which operate on the string data contained in the String object.

```
void setup() {  
  String my_str = "This is my string.";  
  Serial.begin(9600);  
  
  // (1) print the string  
  Serial.println(my_str);  
  
  // (2) change the string to upper-case  
  my_str.toUpperCase();  
  Serial.println(my_str);  
  
  // (3) overwrite the string  
  my_str = "My new string.";  
  Serial.println(my_str);  
  
  // (4) replace a word in the string  
  my_str.replace("string", "Arduino sketch");  
  Serial.println(my_str);  
  
  // (5) get the length of the string  
  Serial.print("String length is: ");  
  Serial.println(my_str.length());  
}  
  
void loop() {  
  
}
```

```
void setup() {  
  String my_str = "This is my string.";  
  Serial.begin(9600);  
  
  // (1) print the string  
  Serial.println(my_str);  
  
  // (2) change the string to upper-case  
  my_str.toUpperCase();  
  Serial.println(my_str);  
  
  // (3) overwrite the string  
  my_str = "My new string.";  
  Serial.println(my_str);  
  
  // (4) replace a word in the string  
  my_str.replace("string", "Arduino sketch");  
  Serial.println(my_str);  
  
  // (5) get the length of the string  
  Serial.print("String length is: ");  
  Serial.println(my_str.length());  
}  
  
void loop() {  
  
}
```

Result

This is my string.

THIS IS MY STRING.

My new string.

My new Arduino sketch.

String length is: 22

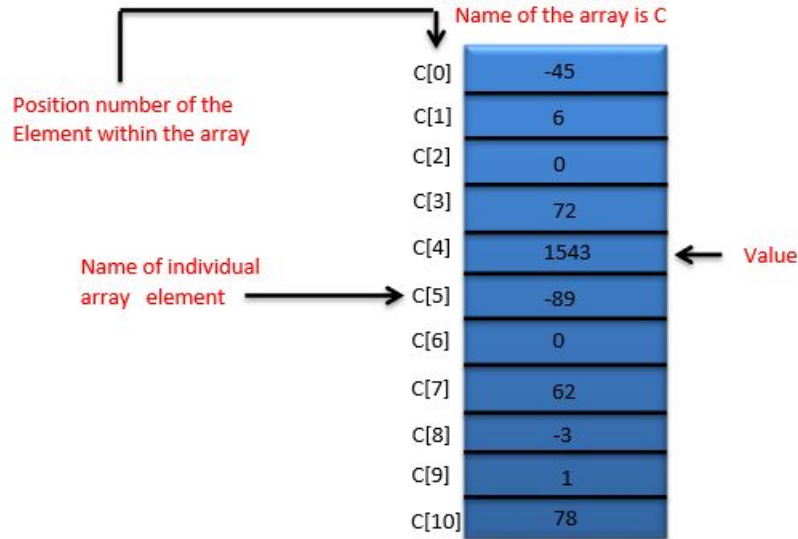
Arduino - Time

Arduino provides four different time manipulation functions. They are –

S.No.	Function & Description
1	delay () function The way the delay() function works is pretty simple. It accepts a single integer (or number) argument. This number represents the time (measured in milliseconds).
2	delayMicroseconds () function The delayMicroseconds() function accepts a single integer (or number) argument. There are a thousand microseconds in a millisecond, and a million microseconds in a second.
3	millis () function This function is used to return the number of milliseconds at the time, the Arduino board begins running the current program.
4	micros () function The micros() function returns the number of microseconds from the time, the Arduino board begins running the current program. This number overflows i.e. goes back to zero after approximately 70 minutes.

Arduino - Arrays

An array is a consecutive group of memory locations that are of the same type. To refer to a particular location or element in the array, we specify the name of the array and the position number of the particular element in the array.



To print the sum of the values contained in the first three elements of array C, we would write –

```
Serial.print (C[ 0 ] + C[ 1 ] + C[ 2 ] );
```

To divide the value of C[6] by 2 and assign the result to the variable x, we would write –

```
x = C[ 6 ] / 2;
```

Declaring an Array and using a Loop to Initialize the Arrays Elements

```
int n[ 10 ] ; // n is an array of 10 integers  
  
void setup () {  
  
}  
  
void loop () {  
  for ( int i = 0; i < 10; ++i ) // initialize elements of array n to 0 {  
    n[ i ] = 0; // set element at location i to 0  
    Serial.print (i) ;  
    Serial.print (\r) ;  
  }  
  for ( int j = 0; j < 10; ++j ) // output each array element's value {  
    Serial.print (n[j]) ;  
    Serial.print (\r) ;  
  }  
}
```


Declaring an Array and using a Loop to Initialize the Arrays Elements

```
int n[ 10 ]; // n is an array of 10 integers  
  
void setup () {  
  
}  
  
void loop () {  
  for ( int i = 0; i < 10; ++i ) // initialize elements of array n to 0 {  
    n[ i ] = 0; // set element at location i to 0  
    Serial.print (i);  
    Serial.print (\r);  
  }  
  for ( int j = 0; j < 10; ++j ) // output each array element's value {  
    Serial.print (n[j]);  
    Serial.print (\r);  
  }  
}
```

Element	Value
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

Initializing an Array in a Declaration with an Initializer List

```
int n[ 10 ] = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };  
  
void setup () {  
  
}  
  
void loop () {  
  for ( int i = 0; i < 10; ++i ) {  
    Serial.print (i);  
    Serial.print (\r);  
  }  
  for ( int j = 0; j < 10; ++j ) // output each array element's value {  
    Serial.print (n[j]);  
    Serial.print (\r);  
  }  
}
```

Initializing an Array in a Declaration with an Initializer List

```
int n[ 10 ] = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };  
  
void setup () {  
}  
  
void loop () {  
  for ( int i = 0; i < 10; ++i ) {  
    Serial.print (i);  
    Serial.print (\r);  
  }  
  for ( int j = 0; j < 10; ++j ) // output each array element's value {  
    Serial.print (n[j]);  
    Serial.print (\r);  
  }  
}
```

Element	Value
0	32
1	27
2	64
3	18
4	95
5	14
6	90
7	70
8	60
9	37

Summing the Elements of an Array

```
const int arraySize = 10; // constant variable indicating size of array  
int a[ arraySize ] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };  
int total = 0;  
  
void setup () {  
  
}  
void loop () {  
    // sum contents of array a  
    for ( int i = 0; i < arraySize; ++i )  
        total += a[ i ];  
    Serial.print (Total of array elements : ) ;  
    Serial.print(total) ;  
}
```

Summing the Elements of an Array

```
const int arraySize = 10; // constant variable indicating size of array  
int a[ arraySize ] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };  
int total = 0;
```

```
void setup () {
```

```
}
```

```
void loop () {
```

```
// sum contents of array a
```

```
for ( int i = 0; i < arraySize; ++i )
```

```
total += a[ i ];
```

```
Serial.print (Total of array elements : ) ;
```

```
Serial.print(total) ;
```

```
}
```

Total of array elements: 849

